

tir el mismo microcódigo, aun cuando realicen operaciones diferentes. Desde luego, las instrucciones PUSH, CALL y JMP son por completo diferentes y cada una requiere de su propio microcódigo.

En la figura 4-33(a) se muestra una microinstrucción típica del 8088 que tiene dos partes que se ejecutan en paralelo. La parte izquierda, que contiene dos campos de 5 bits, permite a cada microinstrucción realizar un movimiento de registro a registro. Los 32 registros que se pueden usar como origen o fuente y destino son los registros de 8 y 16 bits de los dos tableros y los registros temporales.

La parte derecha es una microinstrucción codificada de manera vertical, que consiste en los campos de tipo, código de la ALU, registro y código de condición. Existen seis tipos de microinstrucciones, como se puede ver en la lista de la figura 4-33(b) y se distinguen por el campo TIPO. Todas las microinstrucciones tienen 10 bits para mover un registro fuente a uno de destino, pero el formato de los 11 bits menos significativos varía de un tipo a otro.

El código de la ALU puede ordenar la ejecución de una función específica, o indicar a la ALU que use el código en el registro X. Los 3 bits del campo de registro proporcionan el operando, y el bit final indica si se deberían o no activar los códigos de condición. Cada microinstrucción se ejecuta en un solo ciclo de reloj, de modo que no es posible mover un valor del conjunto de registros internos a un registro TMP y luego utilizar ese registro como un operando de la ALU en el mismo ciclo.

Los saltos largos y las llamadas a microprocedimientos representan un problema al requerir de más bits de los disponibles en la microinstrucción, para designar el destino. La solución es usar una **memoria ROM de traslación** que mapee direcciones de 5 bits en las de todo el microprograma. Los saltos largos y las llamadas utilizan direcciones de 5 bits para indicar el destino que desean. Desde luego, esta estrategia significa que sólo se podrá saltar o llamar hacia 32 direcciones, pero esto es suficiente.

4.6.2. La microarquitectura del Motorola 68000

Como un segundo ejemplo de microarquitectura de una máquina real se verá rápidamente el procesador Motorola 68000 (Stritter y Tredennick, 1978). El 68000 es una pastilla un poco más grande que el 8088, con un espacio potencial para 68000 transistores (de ahí su nombre), aunque sólo se usan aproximadamente 40000, con el resto del espacio ocupado por alambres y demás.

Antes, se ha mencionado el aspecto de si el 68000 es una máquina de 16 o de 32 bits. Esta cuestión se presenta de nuevo en la microarquitectura. En la figura 4-34 se muestra la trayectoria de datos principal.

Como se puede apreciar, la "trayectoria de datos" está formada de hecho por tres trayectorias de datos separadas de 16 bits de ancho, cada una de las

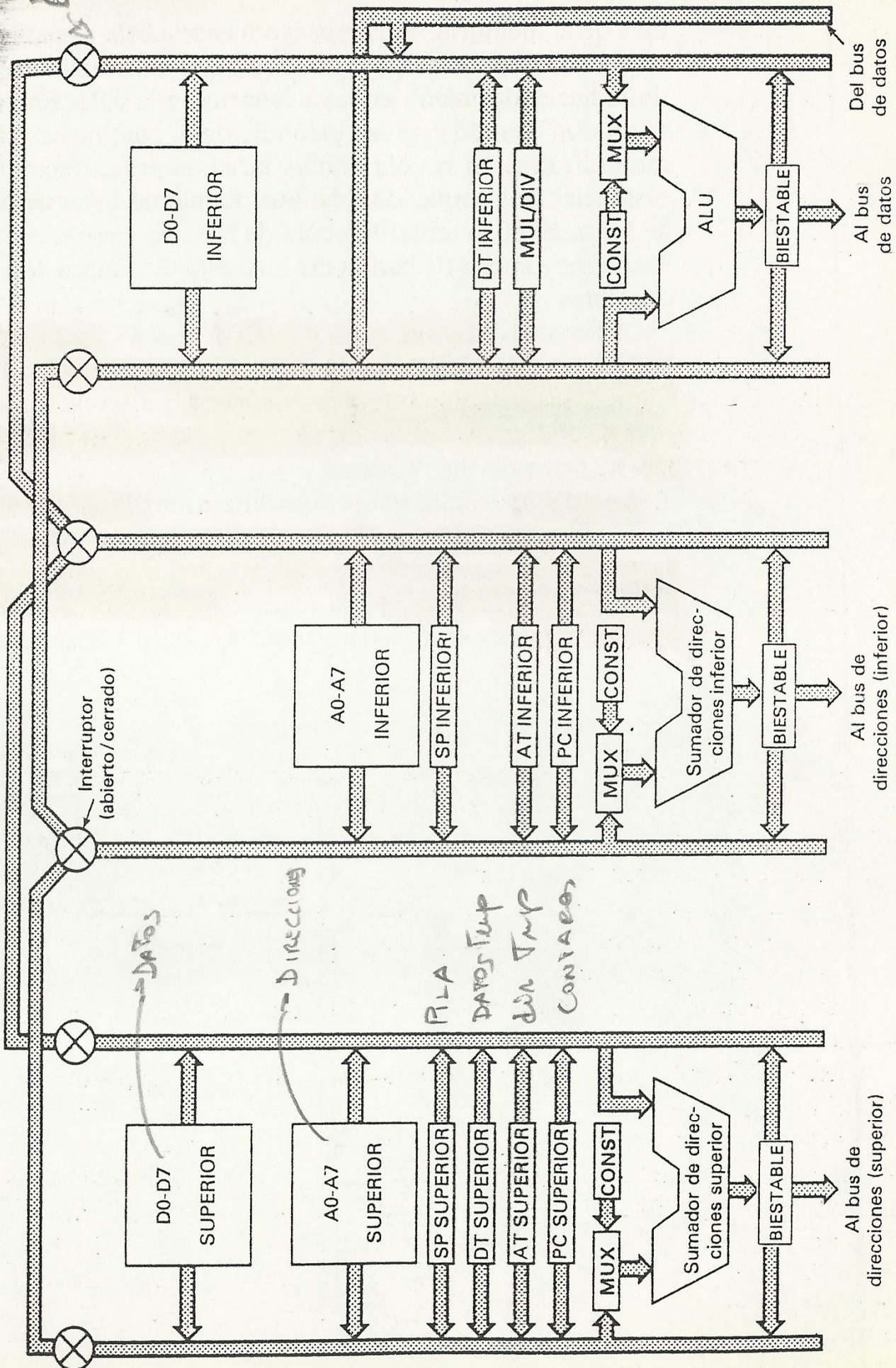


Fig. 4-34. Diagrama sumamente simplificado de las trayectorias de datos del 68000.

cuales puede operar en forma independiente a las otras y en paralelo. La parte izquierda maneja los 16 bits de orden superior de la aritmética de direcciones. La parte del medio se hace cargo de los 16 bits inferiores de las direcciones y la parte derecha realiza operaciones en los datos.

Cada parte tiene dos buses que toman datos de los registros y los alimentan en el sumador o en la ALU. También se usan para llevar la salida de la ALU de regreso a los registros. En la parte superior de cada uno de los buses está un interruptor, que puede abrirse o cerrarse por medio de software. Al abrir el interruptor, los buses pueden conectarse eléctricamente. Si por ejemplo, los seis interruptores se abren, los registros en la parte izquierda se pueden usar como operandos para la ALU en la parte derecha.

Veamos ahora los registros, empezando con los de la izquierda. El recuadro marcado como D0-D7 SUPERIOR contiene los 16 bits de orden superior de los ocho registros de datos. El recuadro bajo el anterior, contiene los 16 bits de orden superior de los ocho registros de direcciones (A0-A7). En seguida está el apuntador de pilas del modo supervisor, dos registros de utilería: TDAT (Datos Temporal) y TDIR (Dirección Temporal) y la mitad superior del contador de programa (CP). Todos estos 20 registros se pueden ubicar, por determinación del microprograma, en el bus izquierdo o en el derecho.

Ambos buses conducen a un sumador de 16 bits. Este no es una ALU completa y sólo puede sumar. En virtud de que se utiliza principalmente para la aritmética de direcciones, no se requiere que haga nada más; construir una ALU completa hubiera empleado mayor área de la pastilla. Nótese que la entrada izquierda del sumador es un multiplexor (MUX), que puede seleccionar el bus de la izquierda o la parte superior de una ROM de constantes (CONST), una vez más, bajo el control del microprograma.

La salida de la ALU tiene un biestable y se puede dirigir a cualquiera de los buses, de modo que los resultados se pueden almacenar en los registros. Además, el biestable puede conducir el bus de dirección externa para salir a la parte superior de una dirección de memoria de 32 bits.

La parte media, es en esencia la misma, excepto que no contiene los registros internos de datos o el registro TDAT. La ROM de constantes contiene, por supuesto, los 16 bits de orden inferior de éstas y no los de orden superior. La salida del biestable puede conducir el bus de direcciones de orden inferior. Es posible habilitar ambas partes en forma simultánea, de manera que se puede poner en el bus de memoria a un tiempo una dirección completa de 32 bits.

La parte de la derecha es similar a las otras dos y contiene la porción de orden inferior de los registros internos de datos y la misma porción de TDAT. También tiene una ROM de constantes, aunque ésta se encuentra en el otro bus y contiene diferentes constantes. Sin embargo, la idea es parecida.

Aparecen en esta parte tres nuevas características. Primero, el sumador ha sido sustituido por una ALU completa, para realizar cálculos aritméticos y fun-

ciones booleanas en los registros D. Ya que la ALU sólo tiene un ancho de 16 bits, las operaciones de 32 bits requieren de dos ciclos de la trayectoria de datos. En segundo lugar, existe un registro extra de utilería (MUL/DIV) que se requiere para las operaciones de multiplicación y división. Y tercero, no sólo envía salidas al bus (de datos), sino que, a diferencia de las otras dos partes, es también posible aceptar datos de entrada.

Vale la pena mencionar, aunque no se muestra en la figura, que existe un conjunto de tres registros para almacenar las instrucciones recién arribadas. Como el 68000 tiene procesamiento en línea (pipeline) se necesita de los tres. El primero de los registros contiene la instrucción que se está ejecutando; el segundo la instrucción que se está decodificando; y el tercero la que se está extrayendo de memoria.

Tampoco se muestra el hecho de que la interfaz contiene interruptores de cruce en ambas direcciones, hacia y desde el bus de datos, como en el 8088, a fin de que los bytes superior e inferior de una palabra de 16 bits puedan intercambiarse mientras van o vienen del bus de datos. Se necesita de esta característica para direccionar y manipular bytes individuales.

El 68000 utiliza un almacén de control de dos niveles con un microprograma y un nanoprograma, como se puede ver en la figura 4-35. Los detalles son en cierto modo diferentes que el ejemplo dado en el texto, pero la meta es la misma: reducir el número de bits en el almacén de control haciendo que el microprograma determine la secuencia de nanoinstrucciones ejecutadas y haciendo, de hecho, que las nanoinstrucciones manejen las compuertas en las trayectorias de datos. El microprograma consiste de 544 palabras de 17 bits y el nanoprograma de 336 palabras de 68 bits, para un total de 32 096 bits. Esta implantación representa un ahorro del 13% sobre otra de un solo nivel con 544 palabras de 68 bits, que hubiera requerido de 36 992 bits. En virtud de que en el momento de introducir esta pastilla al mercado, representaba el estado de arte en la materia, se consideró que dicho ahorro valía la pena. Para fines de comparación, el microprograma del 8088 tiene 10 584 bits en 504 palabras de 21 bits.

Las memorias ROM tanto para el microprograma como para el nanoprograma están direccionadas por números de 10 bits, que permiten hasta 1024 entradas en cada uno de ellos. Como ya se ha mencionado, sólo se requieren 544 para la primera y 336 en la segunda. En ambos casos, las entradas están distribuidas en el espacio de direccionamiento de manera muy cuidadosa. En la nanoprogramación convencional, el hardware extrae primero una microinstrucción, y ésta contiene la dirección de la nanoinstrucción a utilizar. En seguida, se extrae y ejecuta la nanoinstrucción. Este es un proceso inherente de dos etapas, ya que no se puede extraer la nanoinstrucción hasta que esté disponible la microinstrucción.

Los diseñadores del 68000 gustaron de la idea de ahorrar área en la pastilla utilizando la nanoprogramación, pero les desagradó hacer más lenta la má-

No en Motorola

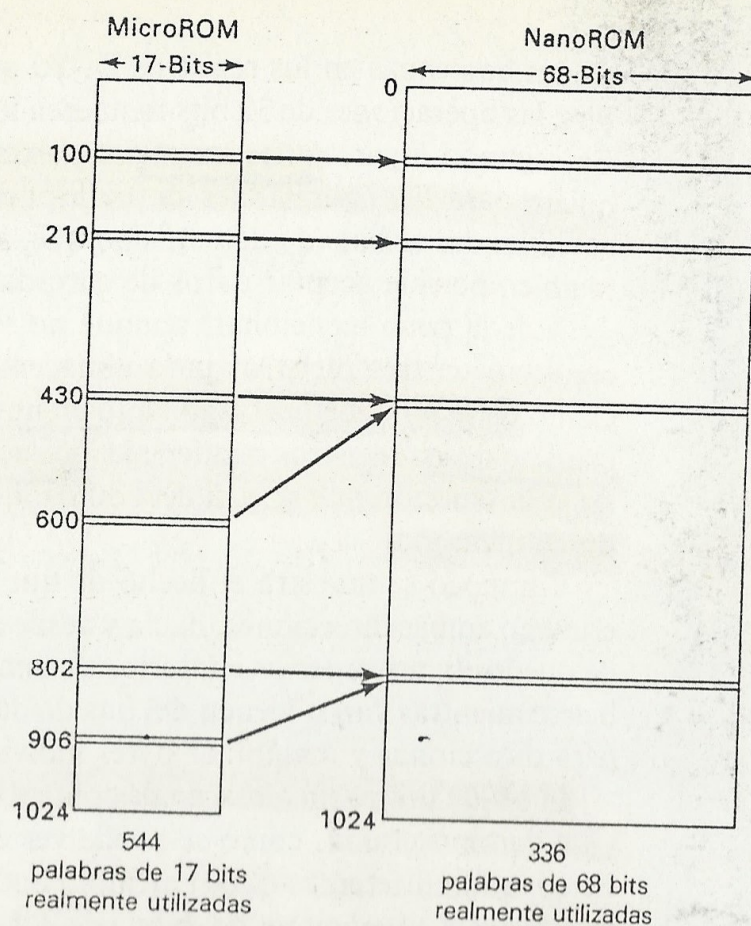


Fig. 4-35. El 68000 tiene un almacén de control de dos niveles con una microROM y una nanoROM.

quina, por tener que esperar a que la microinstrucción estuviera disponible antes de iniciar a extraer la nanoinstrucción. Por ello, utilizaron un truco para hacer todo en un mismo ciclo. En la mayoría de los casos, la nanoinstrucción correspondiente a la microinstrucción que está en la dirección n , se encuentra en la misma dirección, de modo que ambos se pueden extraer al mismo tiempo.

En los casos en que dos o más microinstrucciones comparten una misma nanoinstrucción, se ha quitado un transistor del circuito decodificador de nanoinstrucciones para mapear dos o más direcciones en la misma palabra. Por ejemplo, la dirección 0000011111 se puede mapear en la dirección 0000010111 dejando fuera un transistor, de modo que las microinstrucciones en las direcciones 31 y 23 puedan utilizar ambas la nanoinstrucción 23. Por supuesto, la elección exacta de qué instrucción va donde resulta decisiva, pero los diseñadores tienen computadoras que les ayudan a armar el rompecabezas.

En la figura 4-36 se muestran los dos formatos de microinstrucciones que utiliza el 68000. Las microinstrucciones no se ejecutan en forma secuencial como en el 8088. En vez de ello, cada una nombra a su sucesora. Se requiere de es-

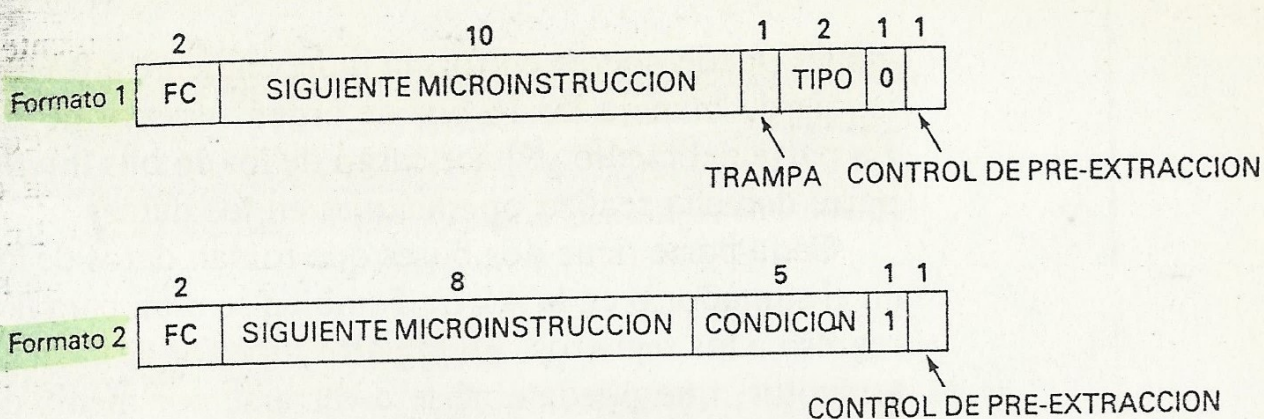


Fig. 4-36. Formatos de instrucciones del 68000.

te esquema dada la necesidad de que microinstrucciones específicas mapeen en determinadas nanoinstrucciones.

El formato 1 se utiliza para todas las microinstrucciones con excepción de los saltos condicionales, los cuales usan el formato 2. Ambos tienen un bit que se emplea para iniciar la pre-extracción de la siguiente instrucción, además tienen en el otro extremo 2 bits (FC), para enviar señales a las patas de la pastilla que indican si una referencia a memoria representa el espacio de instrucciones, el espacio de datos, la recepción de interrupciones, o ninguna de éstas. El formato 1 contiene unos cuantos bits de uso diverso y una dirección de 10 bits que indica de donde extraer la siguiente microinstrucción. El formato 2 tiene un campo de 5 bits que selecciona una de las 32 condiciones preestablecidas para probar, tales como los códigos de condición y bits en el registro de instrucciones. El resultado de la condición produce 2 bits que se combinan con los 8 bits de la microinstrucción para seleccionar uno de los cuatro sucesores posibles.

Las nanoinstrucciones de 68 bits están codificadas en forma horizontal con 38 campos distintos, la mayoría de ellos de uno o dos bits. Los campos controlan las compuertas de todos los registros y las transferencias de los registros internos a los buses, la selección de registros, el control de interruptores, el establecimiento de códigos de condición, la función de la ALU, el uso de memorias ROM de constantes y multiplexores, así como la operación de la memoria. Por lo anterior, se puede apreciar la necesidad de utilizar tantos bits.

4.7. RESUMEN

En su nivel de microprogramación, la CPU consta de dos componentes principales: la ruta de datos y la sección de control. La primera se compone principalmente de una memoria interna y una porción con ALU y registros de corrimiento. Un ciclo consiste en la extracción de algunos operandos de la me-

moria interna, su paso por la ALU y registros de corrimiento, y posiblemente el almacenamiento de los resultados en la memoria interna.

La sección de control consta principalmente de la memoria de control, donde se guarda el microprograma. Cada microinstrucción de la memoria de control controla las compuertas de la ruta de datos durante un microciclo. En el ejemplo dado los microciclos se dividieron en subciclos controlados por un reloj. Durante el primer subciclo se extrajo la microinstrucción de la memoria de control y se puso en un registro interno. En el segundo subciclo se guardaron las entradas de la ALU en sendos registros para mantenerlas estables durante la microinstrucción completa. En el tercer subciclo la ALU y el registro de corrimiento realizan su trabajo. Finalmente, en el cuarto subciclo se guarda el resultado de nuevo en la memoria interna por si se necesitaba en una microinstrucción posterior.

El secuenciamiento de las microinstrucciones puede calificarse de extravagante. Pocas máquinas tienen un contador de microprograma explícito en el nivel de microprogramación. En cambio, las microinstrucciones casi siempre contienen la dirección de base de sus sucesoras. Normalmente se hace el 0 lógico de la dirección base con unos pocos bits de estado para producir la dirección final. En muchas máquinas la dirección base de una microinstrucción se combina con los bits de estado resultante de la anterior para producir una dirección de salto que no tiene efecto sino hasta una microinstrucción después.

Las microinstrucciones se pueden organizar en forma horizontal, con un bit para cada señal de control; verticalmente, con unos cuantos campos que requieren de una decodificación compleja; o a veces en forma combinada.

La organización horizontal conduce a máquinas en paralelo más rápidas y con palabras más grandes. La organización vertical conduce a máquinas más lentas y memorias de control más pequeñas.

Se pueden utilizar varias técnicas para mejorar el desempeño. Una de ellas es la nanoprogramación, en donde el microprograma contiene apuntadores (cortos) hacia nanoinstrucciones (largas) que de hecho controlan las compuertas. La nanoprogramación reduce la cantidad de espacio dedicado a memorias ROM en la pastilla, a costa de una ejecución más lenta.

Otras formas de mejorar el desempeño son: ciclos de tiempo variables, ramificación múltiple, procesamiento en línea y uso de memorias caché. Sin embargo, cada técnica lleva aparejado cierto grado de mayor complejidad.

Por último, se abordaron las microarquitecturas de dos de las pastillas disponibles comercialmente, el 8088 y el 68000. El 8088 tiene una trayectoria de datos, tradicional, aumentada por una sección superior para realizar cálculos en los registros de segmentos y utiliza microinstrucciones verticales de 21 bits. El 68000 tiene tres trayectorias de datos independientes de 16 bits, dos para direcciones y una para datos. Utiliza un control a dos niveles con microinstrucciones de 17 bits y nanoinstrucciones de 68 bits.